

Autômato Finito Determinístico

Definição

Um *autômato finito determinístico* (AFD) M é uma quintupla

$$M := (Q, \Sigma, \delta, q_0, F),$$

onde:

- Q : conjunto finito de *estados*;
- Σ : conjunto finito de *símbolos* — *alfabeto de entrada*;
- $\delta : Q \times \Sigma \rightarrow Q$: *função de transição*;
- $q_0 \in Q$: *estado inicial*;
- $F \subseteq Q$: *estados finais* ou *de aceitação*.

Seja $x \in \Sigma^*$, $|x| = n$. Definimos a função $\Delta : Q \times \Sigma^* \rightarrow Q$ de computação de M , onde

$$\Delta(p, x) = \begin{cases} p & \text{se } x = \epsilon \\ \delta(\Delta(p, x_1 \dots x_{n-1}), x_n) & \text{caso contrário.} \end{cases}$$

Se p é o estado inicial podemos escrever simplesmente $\Delta(x)$ significando $\Delta(p, x)$.

Dizemos que:

1. M *aceita* x se $\Delta(x) \in F$. Denotamos isso por $M(x) = \mathbf{aceita}$;

2. M *rejeita* x se $\neg(M(x) = \text{aceita})$. Denotamos isso por $M(x) = \text{rejeita}$;
3. M *reconhece* a linguagem $L(M) = \{x \in \Sigma^* : M(x) = \text{aceita}\}$;
4. M e N são *equivalentes* se $L(M) = L(N)$. Denotamos isso por $M \equiv N$;
5. a linguagem $A \subseteq \Sigma^*$ é dita *regular* se $A = L(M)$ para algum AFD M .

Representação de um AFD

Podemos representamos um AFD neste texto usando

1. a definição vista na seção anterior;
2. uma tabela representando a função de transição;
3. um diagrama de transição de estados.

Exemplo 1. *Considere o AFD*

$$M := (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_3\})$$

onde $\delta(q_0, a) = q_1$, $\delta(q_1, a) = q_2$, $\delta(q_2, a) = \delta(q_3, a) = q_3$, $\delta(q, b) = q$ para todo $q \in Q$.

Usando uma tabela como na Tabela 1, a “seta” à esquerda de q_0 indica que q_0 é o estado inicial e o F ao lado de q_3 indica que q_3 é um estado final.

		a	b
$M :=$	$\rightarrow q_0$	q_1	q_0
	q_1	q_2	q_1
	q_2	q_3	q_2
	$q_3 F$	q_3	q_3

Tabela 1: Representação de um AFD por uma tabela

Usando um diagrama de transição de estados, a seta apontando para o q_0 indica que q_0 é o estado inicial; e todo estado com círculo duplo é um estado de aceitação. A Figura 1 mostra um diagrama de estados do Exemplo 1.

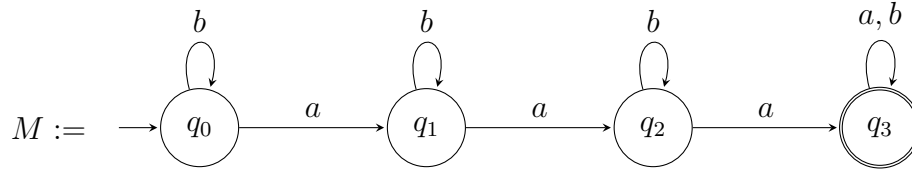
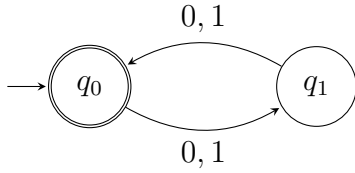


Figura 1: Diagrama de transição de estados do AFD do Exemplo 1.

Exemplo 2. *Descreva um AFD que reconhece a linguagem $\{x \in \{0,1\}^* : |x| \text{ é par}\}$.*

Solução:



Exemplo 3. *Descreva um AFD que reconhece a linguagem*

$$\{x \in \{0,1\}^* : x \text{ começa ou termina com } 01\}.$$

Solução:

O AFD descrito tem 5 estados $q_0, q_1, q_2, q_3, q_4, q_5$ sobre o alfabeto $\{0,1\}$ e função de transição dada pela tabela

δ	0	1
$\rightarrow q_0$	q_1	q_4
q_1	q_3	q_2
q_2^F	q_2	q_2
q_3	q_3	q_5
q_4	q_3	q_4
q_5^F	q_3	q_4

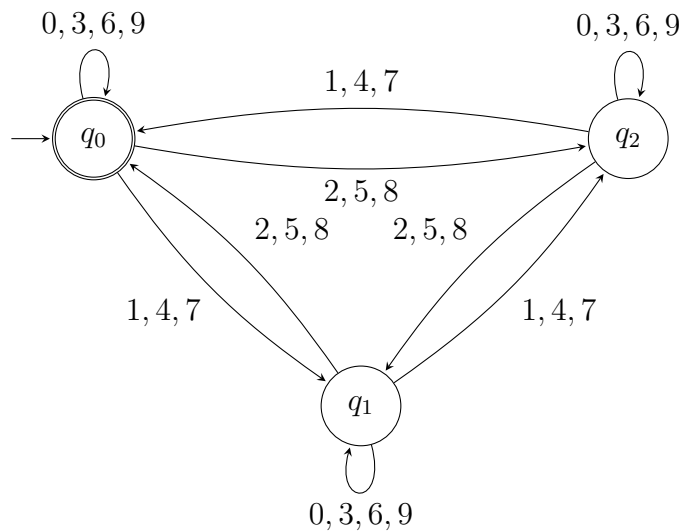
Exemplo 4. *Descreva um AFD que reconhece*

$$\{x \in \{0, \dots, 9\}^* : \text{ o número } x \text{ em decimal é um múltiplo de } 3\}.$$

Solução:

O AFD descrito possui 3 estados q_0, q_1, q_2 sobre o alfabeto $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, e função de transição dada pela tabela

δ	0	1	2	3	4	5	6	7	8	9
$\rightarrow q_0 F$	q_0	q_1	q_2	q_0	q_1	q_2	q_0	q_1	q_2	q_0
q_1	q_1	q_2	q_0	q_1	q_2	q_0	q_1	q_2	q_0	q_1
q_2	q_2	q_0	q_1	q_2	q_0	q_1	q_2	q_0	q_1	q_2



Implementação

Um AFD pode ser visto como um formalismo reconhecedor para representar uma linguagem, ou seja, uma máquina que aceita cadeias de uma determinada linguagem. Esse mecanismo pode ser implementado por um programa de computador. A seguir mostramos um exemplo em python que devolve **True** se uma cadeia s é aceita pelo AFD do Exemplo 1; e devolve **False** caso contrário.

```
def afd (s):
    estados = ['q0', 'q1', 'q2', 'q3']
```

```

alfabeto = ['a', 'b']
transicao = { 'q0' : {'a' : 'q1', 'b' : 'q0'},
              'q1' : {'a' : 'q2', 'b' : 'q1'},
              'q2' : {'a' : 'q3', 'b' : 'q2'},
              'q3' : {'a' : 'q3', 'b' : 'q3'} }
estadoInicial = 'q0'
estadoFinal = ['q3']

q = estadoInicial
i = 0
n = len(s)
while i < n:
    q = transicao[q][s[i]]
    i += 1

return q in estadoFinal

```

Prova de que um AFD reconhece uma linguagem

Vejamos primeiramente o Exemplo 2 e provamos o seguinte invariante.

Invariante 1. *Seja $x \in \Sigma^*$, $n = |x|$. Então,*

$$\Delta(x) = \begin{cases} q_0 & \text{se } n \text{ é par,} \\ q_1 & \text{se } n \text{ é ímpar.} \end{cases}$$

Prova. A prova é por indução em $n = |x|$.

Suponha que $n = 0$. Nesse caso, $x = \epsilon$. Como $\Delta(\epsilon) = q_0$ e $|\epsilon|$ é par, temos que nesse caso vale o invariante.

Suponha então que $n > 0$. Por hipótese de indução, temos que

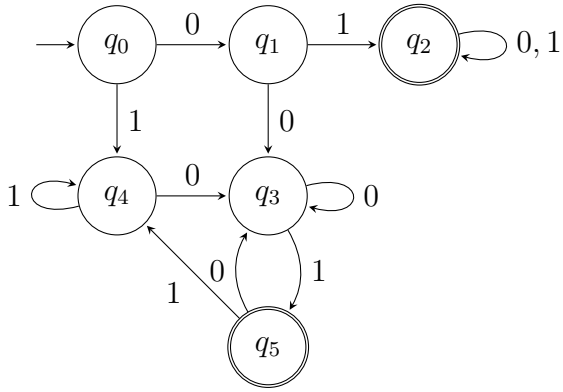
$$\Delta(y = x_1 \dots x_{n-1}) = \begin{cases} q_0 & \text{se } |y| \text{ é par,} \\ q_1 & \text{se } |y| \text{ é ímpar.} \end{cases}$$

Se $|x|$ é par, $|y|$ é ímpar o que implica, por HI que $\Delta(y) = q_1$ e, portanto, $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_1, x_n) = q_0$; similarmente, $\Delta(x) = q_1$ se $|x|$ é ímpar. Logo, vale o

invariante para $n > 0$. □

Desde que $F = \{q_0\}$, segue do invariante acima mostra que M aceita somente as cadeias de comprimento par.

Seja o AFD



Esse AFD reconhece a linguagem $\{x \in \{0, 1\}^* : x \text{ começa ou termina com } 01\}$. Para mostrar que isto é verdade basta mostrar o seguinte invariante.

Invariante 2. *Seja $x \in \{0, 1\}^*$. Então,*

$$\Delta(x) = \begin{cases} q_0 & \text{se } x = \varepsilon, \\ q_1 & \text{se } x = 0, \\ q_2 & \text{se } x \text{ começa com } 01, \\ q_3 & \text{se } x \text{ não começa com } 01 \text{ e termina com } 0, \\ q_4 & \text{se } x \text{ não começa com } 01 \text{ e (é } 1 \text{ ou termina com } 11), \\ q_5 & \text{se } x \text{ não começa com } 01 \text{ mas termina com } 01. \end{cases}$$

Prova. A prova é por indução em $n = |x|$.

Suponha que $n \leq 2$. Nesse caso, $x \in \{\varepsilon, 0, 1, 00, 01, 10, 11\}$. Verificamos que o invariante vale para cada valor possível para x . Portanto, vale o invariante para $n \leq 2$.

Suponha então que $n > 2$. Por hipótese de indução, temos que

$$\Delta(y = x_1 \dots x_{n-1}) = \begin{cases} q_2 & \text{se } y \text{ começa com } 01, \\ q_3 & \text{se } y \text{ não começa com } 01 \text{ e termina com } 0, \\ q_4 & \text{se } y \text{ não começa com } 01 \text{ e termina com } 11, \\ q_5 & \text{se } y \text{ não começa com } 01 \text{ mas termina com } 01. \end{cases}$$

(Note que $y \neq \epsilon$, $y \neq 0$ e $y \neq 1$ porque $|y| \geq 2$.)

Suponha que x começa com 01. Neste caso y começa por 01 e, por HI, $\Delta(y) = q_2$. Logo, $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_2, x_n) = q_2$ o que implica que vale o invariante quando x começa com 01. Assumimos então que x não começa com 01. E claramente y também não começa com 01. Logo, para mostrar que vale o invariante para $n > 2$, basta mostrar que vale consideramos individualmente que: (i) x termina por 00 ou por 01; (ii) x termina por 010 ou por 011; (iii) x termina por 110 ou por 111, abrangendo todos os casos possíveis.

Se x termina por 00 ou por 01, temos que y termina por 0 o que implica, desde que y não começa por 01, que $\Delta(y) = q_3$. Logo, se x termina por 00, temos que $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_3, 0) = q_3$; e se x termina por 01, temos que $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_3, 1) = q_5$ o que implica que vale o invariante se x termina por 00 ou x termina por 01.

Se x termina por 010 ou por 011, temos que y termina por 01 o que implica, desde que y não começa por 01, que $\Delta(y) = q_5$. Logo, se x termina por 010, temos que $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_5, 0) = q_3$; e se x termina por 011, temos que $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_5, 1) = q_4$ o que implica que vale o invariante se x termina por 010 ou por 011.

Se x termina por 110 ou por 111, temos que y termina por 11 o que implica, desde que y não começa por 01, que $\Delta(y) = q_4$. Logo, se x termina por 110, temos que $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_4, 0) = q_3$; e se x termina por 111, temos que $\Delta(x) = \delta(\Delta(y), x_n) = \delta(q_4, 1) = q_4$ o que implica que vale o invariante se x termina por 010 ou por 011. $\square$

Operações Regulares

As seguintes operações sobre linguagens são chamadas *operações regulares*:

1. **União:** $A \cup B = \{x : x \in A \text{ ou } x \in B\}$;
2. **Concatenação:** $A \cdot B (\equiv AB) = \{xy : x \in A \text{ e } y \in B\}$;
3. **Estrela:** $A^* = \{x_1x_2 \dots x_k : k \geq 0 \text{ e cada } x_i \in A\}$.

Seja A um conjunto. Denotamos por 2^A o *conjunto das partes de A* , ou seja o conjunto formado por todos os subconjuntos de A . Reescrevendo,

$$2^A = \{X : X \subseteq A\}.$$

Note que $|2^A| = 2^{|A|}$.

Lema 1. *Seja A uma linguagem regular. Então $A \cup \{\epsilon\}$ também é regular.*

Prova. (Esboço) Como A é regular, existe um AFD $M' = (Q, \Sigma, \delta', r_0, F)$ que reconhece A . Então, o AFD $M = (Q \cup \{q_0\}, \Sigma, \delta, q_0, F \cup \{q_0\})$, $q_0 \notin Q$, tal que para todo $a \in \Sigma$

$$\delta(q, a) = \begin{cases} \delta'(q, a) & \text{para } q \in Q; \\ \delta'(r_0, a) & \text{para } q = q_0 \end{cases}$$

reconhece $A \cup \{\epsilon\}$.

□

Teorema 1. *Sejam A e B linguagens regulares sobre Σ . Então $A \cup B$, $A \cdot B$ e A^* são regulares.*

Prova. (Esboço) Como A e B são regulares, existem AFDs

$$M_A = (Q_A, \Sigma, \delta_A, q_0, F_A) \text{ e } M_B = (Q_B, \Sigma, \delta_B, p_0, F_B)$$

que reconhecem A e B respectivamente. Sem perda de generalidade assumimos $Q_A \cap Q_B = \emptyset$.

O AFD $(Q_A \times Q_B, \Sigma, \delta, (q_0, p_0), \{(q, p) : q \in F_A \text{ ou } p \in F_B\})$ tal que $\delta((q, p), a) = (\delta_A(q, a), \delta_B(p, a))$, para cada $q \in Q_A$ $p \in Q_B$ e $a \in \Sigma$ reconhece $A \cup B$. Portanto, $A \cup B$ é regular.

Seja

$$X(q) = \begin{cases} \{p_0\} & \text{se } q \in F_A, \\ \emptyset & \text{caso contrário} \end{cases}$$

para cada $q \in Q_A$. O AFD $(Q_A \times 2^{Q_B}, \Sigma, \delta, (q_0, X(q_0)), \{(q, S) : S \cap F_B \neq \emptyset\})$ tal que

$$\delta((q, S), a) = (\delta_A(q, a), X(\delta_A(q, a)) \cup \cup_{p \in S} \delta_B(p, a))$$

para cada $q \in Q_A$, $S \subseteq Q_B$ e $a \in \Sigma$ reconhece a linguagem $A \cdot B$. Portanto, $A \cdot B$ é regular.

Seja

$$Y(S) = \begin{cases} \{q_0\} & \text{se } S \cap F_A \neq \emptyset, \\ \emptyset & \text{caso contrário;} \end{cases}$$

O AFD $(2^{Q_A}, \Sigma, \delta, \{q_0\}, \{S : S \subseteq Q_A \text{ e } S \cap F_A \neq \emptyset\})$ tal que $\delta(S, a) = (\cup_{q \in S} \delta_A(q, a)) \cup Y(\cup_{q \in S} \delta_A(q, a))$ para cada $q \in Q_A$, $S \subseteq Q_A$ e $a \in \Sigma$, reconhece A^+ . Usando o Lema 1 segue que $A^* = A^+ \cup \{\epsilon\}$ é regular.

□

Exercícios

1. Construa uma AFD para cada uma das linguagens abaixo.

- (a) $\{w \in \{0, 1\}^* : |w|_0 \geq 3 \wedge |w|_1 \geq 2\}$
- (b) $\{w \in \{0, 1\}^* : |w|_0 = 2 \wedge |w|_1 \geq 3\}$
- (c) $\{w \in \{0, 1\}^* : |w|_0 \bmod 2 == 0 \wedge 1 \leq |w|_1 \leq 2\}$
- (d) $\{w \in \{0, 1\}^* : |w|_0 \bmod 2 == 0 \wedge \text{cada } 1 \text{ é seguido por pelo menos } 2 \text{ zeros.}\}$
- (e) $\{w \in \{0, 1\}^* : |w|_0 \bmod 2 == 1 \wedge w \text{ termina com } 1\}$
- (f) $\{w \in \{0, 1\}^* : |w| \bmod 2 == 0 \wedge |w|_1 \bmod 2 == 1\}$
- (g) $\{w \in \{0, 1\}^* : \exists x \in \{0, 1\}^* (w = 1x0)\}$
- (h) $\{w \in \{0, 1\}^* : |w|_1 \geq 3\}$

- (i) $\{w \in \{0, 1\}^* : \exists x, y \in \{0, 1\}^* (w = x0101y)\}$
- (j) $\{w \in \{0, 1\}^* : |w| \geq 3 \wedge w_3 = 0\}$
2. Descreva um diagrama de estados e implemente um AFD M tal que $L(M)$ seja o conjunto de cadeias $x \in \{0, 1\}^*$ que não terminam por 00 onde $|x|_0$ é par e $|x|_1$ é um múltiplo de 3.
3. Especifique e implemente um AFD M tal que
- $$L(M) = \{x \in \{a, b, c\}^* : \text{todo } b \text{ em } x \text{ é imediatamente seguido por um } c\}.$$
4. Especifique e implemente um AFD M tal que
- $$L(M) = \{x \in \{a, b\}^* : x \text{ tem dois } b\text{'s consecutivos e } x \text{ não tem dois } a\text{'s consecutivos}\}.$$
5. Especifique e implemente um AFD M tal que
- $$L(M) = \{x \in \{a, b\}^* : x \text{ não tem dois } a\text{'s e nem dois } b\text{'s consecutivos}\}.$$
6. Especifique e implemente um AFD M tal que
- $$L(M) = \{x \in \{a, b, c\}^* : x \text{ começa e termina com símbolos distintos}\}.$$
7. Especifique e implemente um AFD M tal que
- $$L(M) = \{x \in \{0, 1, \dots, 9\}^* : \text{a soma dos símbolos de } x \text{ é divisível por } 7\}.$$
8. Especifique e implemente um AFD que aceite exatamente a linguagem consistindo de todas as cadeias x sobre $\{0, 1\}$ tais que a cadeia x interpretada como um número binário seja um múltiplo de 3. Por exemplo, $x = 0$, $x = 000$, $x = 11$, $x = 110$, $x = 001111$ são cadeias da referida linguagem, enquanto $x = 10$ e $x = 001000$ não são.
9. Especifique e implemente um AFD que aceite a linguagem consistindo de todas as cadeias x sobre $\{0, 1, 2\}$ tais que a cadeia x interpretada como um número na base 3 seja um múltiplo de 7.

10. Sejam

$$M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1) \quad \text{e} \quad M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$$

o AFD M_3 definido como segue:

$$M_3 = (Q_3, \Sigma, \delta_3, s_3, F_3)$$

onde

$$\begin{aligned} Q_3 &= Q_1 \times Q_2 = \{(p, q) : p \in Q_1 \text{ e } q \in Q_2\}, \\ F_3 &= F_1 \times F_2 = \{(p, q) : p \in F_1 \text{ e } q \in F_2\}, \\ s_3 &= (s_1, s_2), \end{aligned}$$

e seja

$$\delta_3 : Q_3 \times \Sigma \rightarrow Q_3$$

a função de transição definida por

$$\delta_3((p, q), a) = (\delta_1(p, a), \delta_2(q, a)).$$

O AFD M_3 é denominado o *produto* de M_1 e M_2 . Calcule o AFD produto M_3 dos AFDs M_1 e M_2 dados pelas tabelas abaixo:

		a	b			a	b
$\rightarrow$	q_0	q_0	q_1	$\rightarrow$	q_0	q_1	q_2
	$q_1 F$	q_1	q_0		q_1	q_2	q_0
					$q_2 F$	q_0	q_1

M_1
 M_2

11. Seja

$$M = (Q, \Sigma, \delta, q_0, F)$$

um AFD qualquer. Considere o AFD $\overline{M} = (Q, \Sigma, \delta, q_0, Q - F)$. Então, mostre que $L(\overline{M}) = \Sigma^* - L(M)$.

12. Dado qualquer conjunto X , para quaisquer subconjuntos A e B de X , mostre que $A \subseteq B$ se, e somente se, $A \cap \overline{B} = \emptyset$, onde $\overline{B}$ é o complemento de B em relação a X .

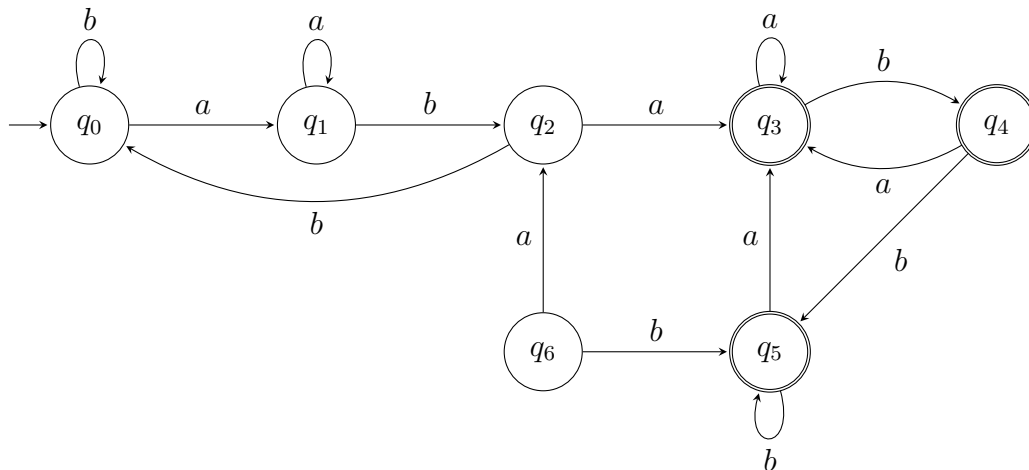
13. Dados dois AFDs, M_1 e M_2 , escreva um algoritmo para decidir se $L(M_1) \subseteq L(M_2)$. *Dica: considere o AFD produto.*

14. Seja $M = (Q, \Sigma, \delta, q_0, F)$ um AFD. Forneça uma condição suficiente em M para que $\epsilon \in L(M)$. A sua condição é também necessária? Justifique sua resposta.
15. Seja $M = (Q, \Sigma, \delta, q_0, F)$ um AFD qualquer. Especifique e implemente um outro AFD M' , tal que $L(M') = L(M) - \{\epsilon\}$.
16. Prove que se L é uma linguagem regular, então $L^{\mathbf{R}}$ é uma linguagem regular.
17. Mostre que se A e B são linguagens regulares, então $A \cap B$ é regular.
18. Mostre que se A é regular, então $\overline{A}$ é regular.
19. Seja A e B linguagens tais que A é regular e $\overline{B}$ é regular. Mostre que $A \cup (A \cdot B)$ é regular.

Minimização de Estados

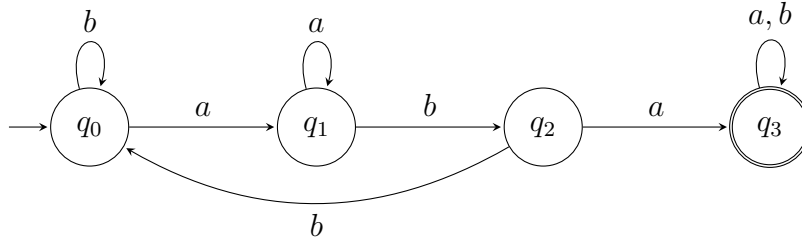
Considerações iniciais

Considere o AFD da figura abaixo.



Note que os estados q_3 , q_4 e q_5 poderiam ser “fundidos” em um único estado, pois eles são todos estados finais e, uma vez que o AFD entre em um deles, ele não pode mais sair. Feito isto, note agora que o estado q_6 não pode ser alcançado, de modo

que a sua presença não influencia a aceitação de qualquer cadeia o que significa que ele pode ser excluído. Portanto, o AFD ilustrado é equivalente ao AFD

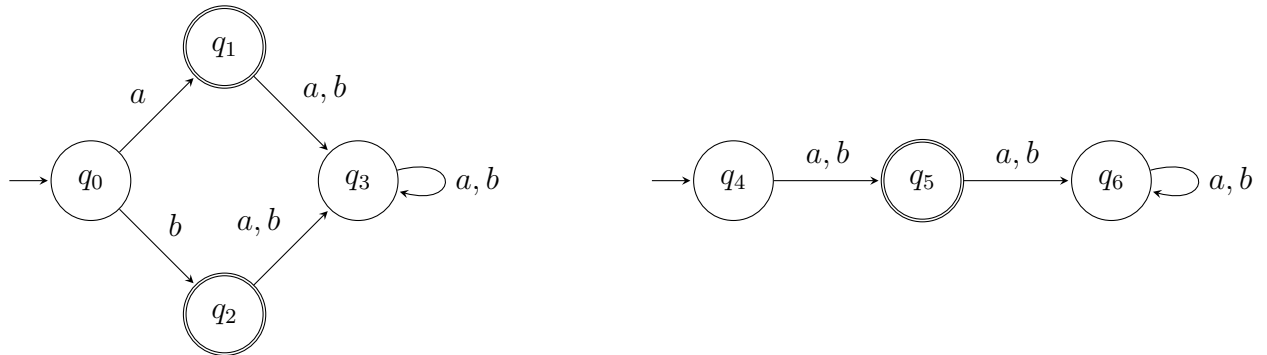


Então, dada uma linguagem regular A , uma questão interessante (na verdade importante) é como podemos encontrar um AFD M tal que $L(M) = A$ e M tenha o menor número de estados entre todos os AFDs que reconhecem A ? Este processo é denominado *minimização de estados* e consiste de duas etapas:

1. eliminar *estados inatingíveis*; isto é, eliminar os estados $q \in Q$ para os quais não existe $x \in \Sigma^*$ tal que $\Delta(x) = q$;
2. fundir estados “*equivalentes*”.

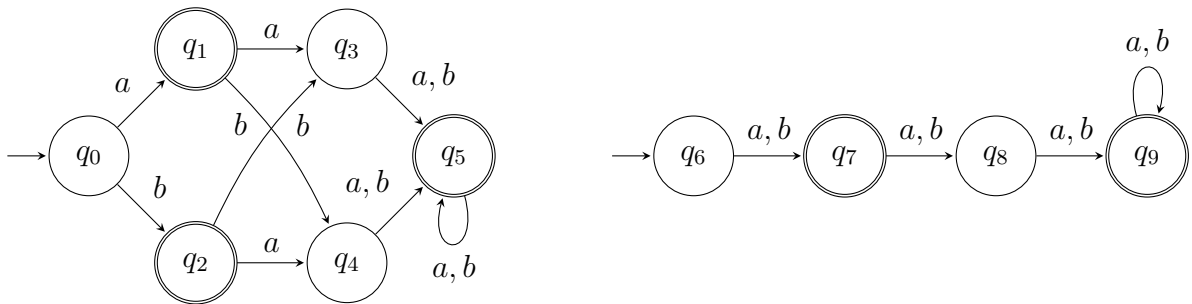
A etapa de eliminação de estados inatingíveis não altera a linguagem reconhecida pelo AFD e pode ser efetuada por um algoritmo simples baseado em uma busca em profundidade no “grafo” correspondente ao diagrama de transição do AFD. Portanto, vamos assumir que esta etapa tenha sido realizada. Para a etapa 2, nós precisamos definir claramente o que significa estado equivalente e como nós podemos fundir dois deles em um só. Para tal, vamos primeiro dar uma olhada na série de exemplos dada a seguir.

Exemplo 5. Considere os dois AFDs abaixo:



Ambos reconhecem a mesma linguagem $\{a, b\}$. O AFD com 4 estados entra em estados distintos dependendo do primeiro símbolo lido, mas não há razão nenhuma para que os estados destinos sejam distintos. Eles são “equivalentes” e podem ser fundidos em um só estado, dando origem ao AFD com 3 estados.

Exemplo 6. Considere os dois AFDs abaixo:

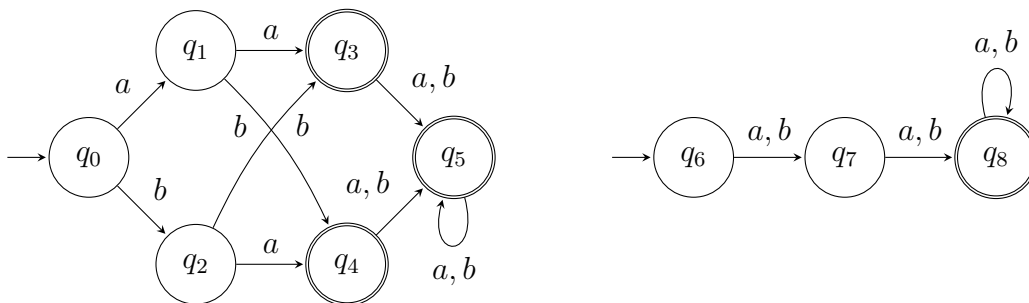


Ambos reconhecem a mesma linguagem, $\{x : |x| = 1 \vee |x| \geq 3\}$. No AFD com mais estados, os estados q_3 e q_4 são equivalentes, pois ambos possuem transições para o estado q_5 para todos os símbolos de entrada. Logo, não há razão para eles serem distintos. Uma vez que q_3 e q_4 são fundidos, nós também podemos fundir q_1 e q_2 pela mesma razão, dando origem ao AFD com menos estados.

Exemplo 7. Considere os dois AFDs a seguir. Ambos reconhecem a mesma linguagem:

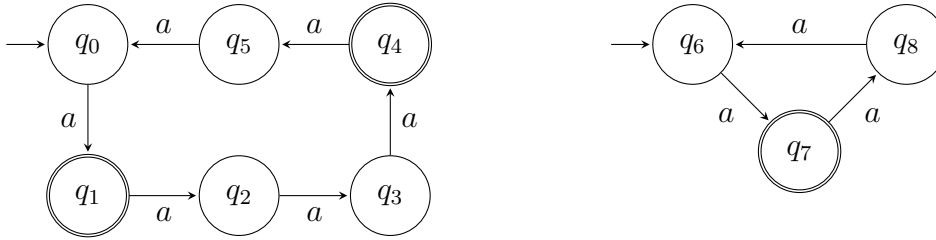
$$\{x \in \{a, b\}^* : |x| \geq 2\}.$$

Os estados q_1 e q_2 são equivalente e podem ser fundidos; similarmente q_3 , q_4 e q_5 também são equivalentes e também podem ser fundidos em um único estado, dando origem ao AFD com menos estados.



Exemplo 8. Os dois AFDs a seguir reconhecem a mesma linguagem:

$$\{a^n : (n - 1) \text{ é um múltiplo de } 3\}.$$



No AFD com mais estados, os estados q_1 e q_4 são equivalentes e podem ser fundidos, dando origem ao AFD com menos estados.

O AFD mínimo

Como sabemos em geral quando dois estados podem ser fundidos em um só sem mudar a linguagem reconhecida pelo AFD original? Como nós sabemos quando não podemos mais fundir estados de um dado AFD?

Considerando que todos os estados do AFD M são atingíveis, considere dois estados: p e q , tais que $\delta(p, \epsilon) \in F$ e $\delta(q, \epsilon) \notin F$. Será que podemos fundir p e q em um único estado? Como $\delta(p, \epsilon) = F$ e $\delta(q, \epsilon) \notin F$, temos que $p \in F$ e $q \notin F$, o que implica que $p = \Delta(x)$ e $q = \Delta(y)$, ou seja, $x \in L(M)$ e $y \notin L(M)$. Fundir os estados e considerar que o estado resultante está em F é considerar que $y \in L(M)$ o que é uma contradição; e considerar que o estado resultante não está em F é considerar que $x \notin L(M)$ o que também é contradição. Logo, não podemos fundir um estado de aceitação com um de não aceitação. A seguir vamos mostrar quando p e q podem ser fundidos.

Primeiro, vamos definir uma relação em Q , denotada por $\sim$, como segue:

$$p \sim q \iff \forall x \in \Sigma^* (\Delta(p, x) \in F \leftrightarrow \Delta(q, x) \in F).$$

e dizemos que p e q são *equivalentes*. Não é difícil verificar que a relação $\sim$ é de fato uma relação de equivalência, isto é, possui as propriedades *reflexiva* ($p \sim p$ para todo $p \in Q$); *simétrica* ($p \sim q \rightarrow q \sim p$ para todo $p, q \in Q$); e *transitiva*

($p \sim q$ e $q \sim r \rightarrow p \sim r$ para todo $p, q, r \in Q$). Logo $\sim$ particiona Q em *classes de equivalência*:

$$[p] = \{q \in Q : q \sim p\}.$$

Nós agora definimos um AFD, $M_{\min}$, chamado *AFD mínima* de M , tal que os estados de $M_{\min}$ correspondem às classes de equivalência de $\sim$:

Seja

$$M_{\min} = (Q', \Sigma, \delta', s', F'),$$

onde

- $Q' = \{[p] : p \in Q\},$
- $\delta'([p], a) = [\delta(p, a)], \forall [p] \in Q, a \in \Sigma,$
- $s' = [s],$
- $F' = \{[p] : p \in F\}.$

O lema a seguir mostra que a função δ' está *bem definida*.

Lema 2. *Sejam $p, q \in Q$ e $a \in \Sigma$. Se $p \sim q$, então $\delta(p, a) \sim \delta(q, a)$.*

Prova. Seja $x \in \Sigma^*$ e suponha que $p \sim q$. Segue que $\forall x (\Delta(p, ax) \in F \leftrightarrow \Delta(q, ax) \in F)$. Como $\Delta(p, ax) = \Delta(\delta(p, a), x)$ e $\Delta(q, ax) = \Delta(\delta(q, a), x)$, segue que $\forall x (\Delta(\delta(p, a), x) \in F \leftrightarrow \Delta(\delta(q, a), x) \in F)$. Segue da definição que $\delta(p, a) \sim \delta(q, a)$. $\square$

O Lema 2 mostra que $[p] = [q]$, então $[\delta(p, a)] = [\delta(q, a)]$ o que significa que a função δ' está bem definida. O conjunto F' também está bem definido e, portanto, segue claramente da definição de F' que

Os seguintes resultados provam que $L(M_{\min}) = L(M)$:

Lema 3. $p \in F \iff [p] \in F'.$

Lema 4. *Para toda cadeia $x \in \Sigma^*$, $\Delta'([p], x) = [\Delta(p, x)]$.*

Prova. Indução em $n = |x|$.

Suponha que $n = 0$. Nesse caso $x = \epsilon$, o que implica que

$$\Delta'([p], x) = \Delta'([p], \epsilon) = [p] = [\Delta(p, \epsilon)] = [\Delta(p, x)].$$

Suponha agora que $n > 0$. Logo, $x \in \Sigma^n$. Como $n > 0$, nós podemos escrever $x = ya$, para algum $y \in \Sigma^{n-1}$ e algum $a \in \Sigma$. Então,

$$\begin{aligned} \Delta'([p], x) &= \Delta'([p], ya) \\ &= \delta'(\Delta'([p], y), a) && \text{(pela definição de } \Delta') \\ &= \delta'([\Delta(p, y)], a) && \text{(pela hipótese de indução)} \\ &= [\delta(\Delta(p, y), a)] && \text{(pela definição de } \delta') \\ &= [\Delta(p, ya)] && \text{(pela definição de } \Delta) \\ &= [\Delta(p, x)]. \end{aligned}$$

□

Teorema 2. $L(M_{\min}) = L(M)$.

Prova. Para qualquer $x \in \Sigma^*$,

$$\begin{aligned} x \in L(M_{\min}) &\iff \Delta'(s', x) \in F' && \text{(definição de aceitação)} \\ &\iff \Delta'([s], x) \in F' && \text{(definição de } s') \\ &\iff [\Delta'(s, x)] \in F' && \text{(Lema 4)} \\ &\iff \Delta(s, x) \in F && \text{(Lema 3)} \\ &\iff x \in L(M) && \text{(definição de aceitação)}. \end{aligned}$$

□

Nós acabamos de provar que $M_{\min}$ é equivalente a M . Agora, é natural nos perguntarmos se $M_{\min}$ é o “menor” AFD equivalente a M que podemos construir removendo estados inatingíveis e fundindo estados equivalentes. A resposta é sim. Para provar

este fato, vamos usar a construção do AFD mínimo em $M_{\min}$ para tentar fundir dois estados quaisquer de $M_{\min}$, dando origem a um AFD “menor”.

Seja

$$[p] \sim [q] \iff \forall x \in \Sigma^* (\Delta'([p], x) \in F' \iff \Delta'([q], x) \in F').$$

A relação acima é a mesma que $\sim$, mas ela é definida no conjunto de estados Q' do AFD mínimo $M_{\min}$. Agora,

$$[p] \sim [q] \Rightarrow \forall x \in \Sigma^* (\Delta'([p], x) \in F' \iff \Delta'([q], x) \in F') \quad (\text{definição de } \sim)$$

$$\Rightarrow \forall x \in \Sigma^* ([\Delta(p, x)] \in F' \iff [\Delta(q, x)] \in F') \quad (\text{pelo Lema 4})$$

$$\Rightarrow \forall x \in \Sigma^* (\Delta(p, x) \in F \iff \Delta(q, x) \in F) \quad (\text{pelo Lema 3})$$

$$\Rightarrow p \sim q \quad (\text{definição de } \sim)$$

$$\Rightarrow [p] = [q].$$

Logo, quaisquer dois estados equivalentes de $M_{\min}$ são de fato iguais, e a relação $\sim$ em Q' nada mais é do que a relação identidade $=$. Isto significa que $M_{\min}$ é o menor AFD que se pode construir através da remoção de estados inatingíveis e da fundição de estados equivalentes de M .

Um Algoritmo para Minimização de Estados

Agora, nós estudaremos um algoritmo para descobrir os pares de estados equivalentes de um dado AFD M . Tal algoritmo é denominado *algoritmo de minimização de estados* e nos permite construir o AFD minimal $M_{\min}$, que reconhece $L(M)$. O algoritmo assume que não há estados inatingíveis em M . Isto não chega a ser uma restrição, pois nós podemos remover tais estados usando um simples algoritmo como mencionado antes.

A operação básica do algoritmo é marcar pares $\{p, q\}$ (não ordenados) de estados de M tão logo esse algoritmo descubra que p e q *não* são equivalentes. Dois estados p, q são equivalentes se vale que $\Delta(p, x)$ é um estado final se e somente se $\Delta(q, x)$ é um estado final para todo $x \in \Sigma^*$. Se p e q são equivalentes escrevemos $p \sim q$.

Os passos do algoritmo são os seguintes:

1. construa uma tabela tal que cada entrada da tabela corresponde a um par não ordenado $\{p, q\}$ de estados de M . Todos os pares estão inicialmente desmarcados. A informação se um par de estados está marcado ou não é armazenada na entrada da tabela correspondente ao par;
2. marque $\{p, q\}$ se $p \in F$ e $q \notin F$ ou vice-versa;
3. repita o seguinte procedimento até que ele não marque mais nenhum par de estados: se existir um par $\{p, q\}$ desmarcado tal que $\{\delta(p, a), \delta(q, a)\}$ está marcado para algum $a \in \Sigma$, então marque $\{p, q\}$.

Quando o algoritmo terminar, nós temos que $p \sim q$ se, e somente se, $\{p, q\}$ não está marcado. Logo, os estados equivalentes entre si que não estão marcados devem ser fundidos para gerar os estados de $M_{\min}$.

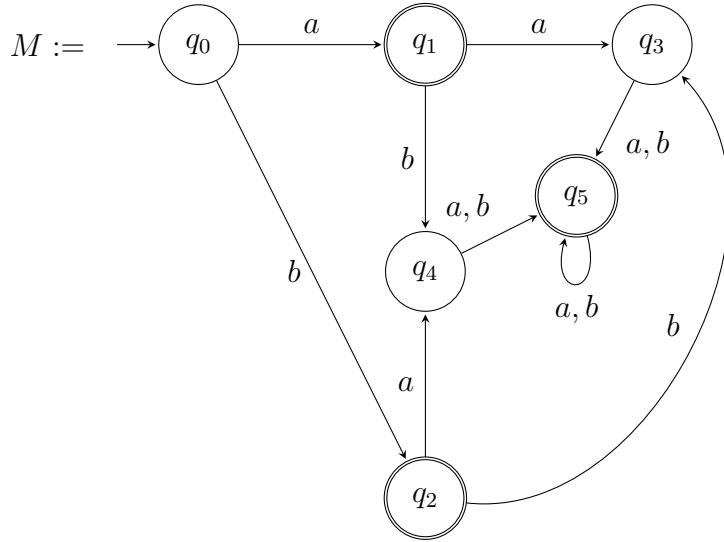
Exemplo 9. Vamos aplicar o algoritmo de minimização acima ao AFD M ,

$$M = (Q, \Sigma, \delta, s, F)$$

tal que $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$, $\Sigma = \{a, b\}$, $s = q_0$, $F = \{q_1, q_2, q_5\}$ e δ é dada pela tabela a seguir:

δ	a	b
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_4	q_3
q_3	q_5	q_5
q_4	q_5	q_5
q_5	q_5	q_5

O diagrama de transição de M é mostrado a seguir:



Primeiramente assinalamos os pares de estado pq tais que $p \in F$ e $q \neq F$.

q_1	X				
q_2	X				
q_3		X	X		
q_4		X	X		
q_5	X			X	X
	q_0	q_1	q_2	q_3	q_4

Depois marcamos q_5q_1 e q_5q_2 pois $\delta(q_5, a) = q_5 \not\sim q_1 = \delta(q_2, a) = \delta(q_2, a)$.

q_1	X				
q_2	X				
q_3		X	X		
q_4		X	X		
q_5	X	X	X	X	X
	q_0	q_1	q_2	q_3	q_4

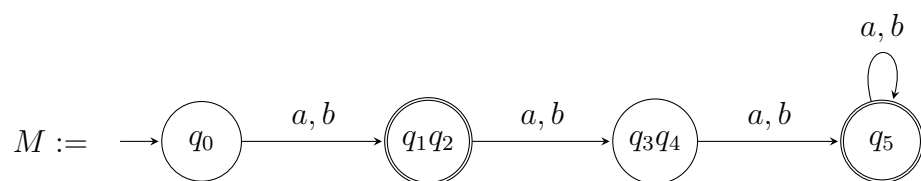
Depois marcamos q_0q_3 e q_0q_4 pois $\delta(q_0, a) = q_1 \not\sim q_3 = \delta(q_3, a) = \delta(q_4, a)$.

q_1	X				
q_2	X				
q_3	X	X	X		
q_4	X	X	X		
q_5	X	X	X	X	X
	q_0	q_1	q_2	q_3	q_4

Agora não marcamos mais nada

$$\begin{aligned} \delta(q_1, a) &\sim \delta(q_2, a) \quad e \quad \delta(q_1, b) \sim \delta(q_2, b); \\ \delta(q_3, a) &\sim \delta(q_4, a) \quad e \quad \delta(q_3, b) \sim \delta(q_4, b); \end{aligned}$$

*e o algoritmo termina fazendo fundindo os estados q_1q_2 ; e fundindo os estados q_3q_4 .
O diagrama de estados resultante é:*



Referências Bibliográficas